Amazon Applied Scientist

Interview Preparation Guide

Problem Framing · Data Understanding · Modeling · Evaluation · Tradeoffs · Pitfalls
5 Fully Solved Case Studies: Recommendations · Search · Fraud · A/B Testing · LLM/RAG

Table of Contents

SECTION 1 — Problem Framing Framework: The MLDP Method

The MLDP Method is a 6-step scaffold for structuring any open-ended ML case study answer. Walk through every step in order — interviewers grade on structure as much as content.

M — Mission & Business Goal

Restate the business objective in one sentence. Identify the north-star business metric (revenue, engagement, retention). Clarify scope: online vs batch, user-facing vs internal.

L — Label / Output Definition

Define what the model predicts: binary label, multi-class, ranking score, continuous value, sequence. Identify the unit of prediction (user, item, user-item pair, query-doc pair).

D — Data Sources

Enumerate available signals: user logs, item catalog, interaction history, 3rd-party feeds. Assess volume, freshness, and coverage. Flag missing data risk early.

P — Pipeline Architecture

Sketch the system: data ingestion → feature store → training → serving. Decide: real-time vs batch inference, online learning feasibility, cold-start handling.

— Modeling Strategy

Propose baseline → simple model → production model. Justify model family with tradeoffs. For LLMs: discuss fine-tuning vs RAG vs prompt engineering.

— Metrics, Monitoring & Guardrails

Select offline and online metrics. Define success criteria. Add guardrails (fairness, latency SLA, business constraints). Plan shadow mode / gradual rollout.

■ FRAMING TIP — Always clarify before modeling

In interviews, say: 'Before I propose a model, let me confirm the objective. Are we optimizing for clicks, purchases, or long-term retention? The answer changes the label definition and the loss function.'

SECTION 2 — Data Understanding: The 5-D Framework

Systematically audit data before proposing features or models. Use the 5-D checklist:

Dimension	What to Check
Distribution	Check label imbalance, feature skew (log-transform heavy-tailed). Plot histograms. Identify outliers (3σ rule, IQR).
Density / Coverage	What fraction of user-item pairs have labels? Sparse matrices need specialized models (ALS, BPR). Coverage < 1% → cold-start problem.
Drift	Is data IID? Check temporal drift: model trained in Jan may degrade by July. Use feature drift monitoring (PSI, KL-divergence). Plan periodic retraining.
Dependency / Leakage	Identify features computed from the future (target leakage). Check train/test splits respect time order. Remove post-event features.
Diversity & Bias	Is training data representative? Selection bias: only logged impressions, not all items. Popularity bias: head items dominate. Correct with inverse propensity scoring (IPS).

Feature Engineering Taxonomy

Feature Type	Examples
User Features	Age, location, historical CTR, purchase history, lifetime value, days since last visit
Item Features	Category, price, avg rating, review count, freshness, brand embeddings
Contextual	Time of day, device type, session position, recent query
Interaction	User-item cross features (user bought same brand before), co-occurrence counts
Embedding-based	User/item ID embeddings (learned), BERT text embeddings, image CNN features
Temporal	Rolling 7/30/90-day aggregates, trend features, seasonality indicators

Handling Missing Data

Missing Type	Strategy
MCAR (Missing Completely at Random)	Mean/median imputation safe. Low risk of bias.
MAR (Missing at Random)	Model-based imputation (kNN, MICE). Add missingness indicator feature.
MNAR (Missing Not at Random)	Missingness is informative — encode as feature. Example: no price listed → likely sold out.

SECTION 3 — Modeling Approaches: Classical ML + LLMs

3.1 Classical ML Model Families

Model	Best For	Pros	Cons
Logistic Regression	Binary classification baseline	Fast, interpretable, calibrated	No feature interactions, linear boundary
Gradient Boosted Trees (XGBoost, LightGBM)	Tabular ranking/classification	Handles mixed features, fast training	Slow inference at scale, no native sequences
Factorization Machines	Sparse feature interactions (rec systems)	Learns pairwise interactions, O(kd) inference	Shallow interactions only
Deep Neural Networks (MLP, DCN)	Dense feature spaces, embeddings	Arbitrary non-linearities, end-to-end	Data hungry, black box
Two-Tower (DSSM)	Retrieval / ANN search	Scalable: pre-compute item embeddings	No interaction between query & item during encoding
Transformer (BERT, T5)	NLP tasks, semantic search	State-of-art NLU, transfer learning	Expensive to serve, latency
LightFM / ALS / BPR	Collaborative filtering	Works with implicit feedback	No content features natively
Isolation Forest / Autoencoders	Anomaly / fraud detection	Unsupervised, no labels needed	High FPR, needs threshold tuning

3.2 Retrieval-Augmented Generation (RAG) Architecture

Step	Details
1. Ingestion	Chunk documents (512 tokens, 10% overlap). Embed with dense encoder (e.g., text-embedding-ada-002, BGE). Store in vector DB (FAISS, Pinecone, OpenSearch k-NN).
2. Retrieval	Encode query. ANN search (HNSW graph, ef=128). Retrieve top-K (K=10–50). Optional: hybrid BM25 + dense (RRF fusion).
3. Re-ranking	Cross-encoder re-ranker on top-K to top-k' (k'=3–5). Captures query-doc interaction. Much more accurate than bi-encoder.
4. Generation	Inject top-k' passages into LLM prompt as context. LLM generates grounded answer. Add citations for faithfulness.
5. Guardrails	Faithfulness check (NLI model: does answer follow context?). Hallucination detection. PII filtering. Latency budget per step.

3.3 LLM Fine-Tuning Decision Tree

	When to Use	Tradeoff	
ering	General tasks, fast iteration, no labeled data	Zero cost, zero latency overhead	Limited customization, context-window
	Knowledge-intensive Q&A;, dynamic knowledge base	No retraining, updatable knowledge	Retrieval quality is a ceiling
Context	Task adaptation without training	No GPU, flexible	Eats context window, inconsistent
	Task-specific adaptation, limited data	Efficient (< 1% params trained)	Needs labeled data (1k–100k example
g	Domain adaptation, new capabilities	Maximum performance	Expensive, catastrophic forgetting risk
	Alignment, preference optimization	Matches human preferences	Needs preference data, complex pipeli

SECTION 4 — Evaluation Metrics (Detailed)

4.1 Classification Metrics

Metric	Formula / Definition	Range	Use When
Accuracy	TP+TN / (TP+TN+FP+FN)	[0,1]	Only when classes are balanced
Precision	TP / (TP + FP)	[0,1]	Cost of false positives is high (spam filter)
Recall (Sensitivity)	TP / (TP + FN)	[0,1]	Cost of false negatives is high (fraud, cancer)
F1-Score	2·P·R / (P+R) — harmonic mean	[0,1]	Imbalanced classes, both errors matter
F-beta Score	$(1+\beta^2)\cdot P\cdot R / (\beta^2\cdot P+R)$	[0,1]	$\beta>1$: recall matters more; $\beta<1$: precision matters more
AUC-ROC	Area under TPR vs FPR curve	[0.5,1]	Ranking quality, class-imbalanced. Threshold-independent.
AUC-PR	Area under Precision-Recall curve	[0,1]	Very imbalanced datasets (fraud: 0.1% positive)
Log-Loss	$-(y\cdot\log(p)+(1-y)\cdot\log(1-p))$	[0,∞)	When calibrated probabilities needed
Calibration (ECE)	Expected Calibration Error	[0,1]	Risk scoring, bid pricing — need trustworthy probabilities
MCC	(TP·TN–FP·FN)/√((TP+FP)⋯)	[–1,1]	Most robust single metric for binary imbalanced

4.2 Regression Metrics

Metric	Formula / Definition	Range	Use When
MAE	mean(|y – ■|)	[0,∞)	When outliers shouldn't dominate (median-like)
RMSE	√mean((y–■)²)	[0,∞)	Penalizes large errors; use for forecasting
MAPE	mean(|y–■|/y)×100%	[0,∞)%	Scale-independent comparison across products
R^2 (CoD)	1 – SS_res/SS_tot	(–∞,1]	Proportion of variance explained
Huber Loss	Quadratic <δ, linear ≥δ	[0,∞)	Robust to outliers, differentiable

4.3 Ranking & Recommendation Metrics

These are critical for search, rec systems, and LLM retrieval. Understand the formulas and intuitions deeply.

Core Ranking Metrics

Metric	Formula / Definition	Range	Use When
Precision@K	# relevant in top-K / K	[0,1]	When all top-K slots equally important
Recall@K	# relevant in top-K / total relevant	[0,1]	When coverage matters (medical search)
Hit Rate@K (HR@K)	Fraction of users with ≥1 relevant in top-K	[0,1]	Next-item prediction tasks
MRR (Mean Reciprocal Rank)	mean(1/rank of first relevant item)	[0,1]	When only first relevant result matters (Q&A;)
AP (Average Precision)	mean of Precision@k for each relevant k	[0,1]	Ordered relevance list evaluation
MAP (Mean AP)	mean of AP across queries	[0,1]	Overall system ranking quality
DCG@K	Σ (rel_i / log■(i+1)) for i=1..K	[0,∞)	Graded relevance, position-weighted
NDCG@K	DCG@K / IDCG@K (ideal DCG)	[0,1]	Normalized; compare across queries
AUC (Ranking)	P(score(pos) > score(neg))	[0.5,1]	Pairwise ranking quality

■ NDCG Deep Dive

NDCG@K is the gold-standard ranking metric. The key insight: **position matters**. Showing a relevant item at rank 1 is much more valuable than rank 5.

```
DCG@K = Σ■■■■ (2^rel■ − 1) / log■(i+1)
NDCG@K = DCG@K / IDCG@K where IDCG = DCG of perfect ranking
```

Amazon uses NDCG extensively in search and recommendations. **Pitfall:** NDCG requires graded relevance labels (0–4). Binary labels reduce it to a weaker signal.

4.4 A/B Testing & Statistical Inference Metrics

Concept	Definition / Formula	Key Insight
Type I Error (α)	False positive rate — rejecting H■ when true. Typically α = 0.05.	Set before experiment. Lower α = fewer false discoveries but need larger N.
Type II Error (β)	False negative rate — failing to reject H■ when false.	Power = 1–β. Usually target power = 0.80.
p-value	Probability of observing data this extreme under H■.	p < α → reject H■. NOT the probability H■ is true.

Effect Size (Cohen's d)	$d = (\mu_B - \mu_A) / \sigma_{pooled}$	Standardized magnitude. |d|: small=0.2, medium=0.5, large=0.8.
Confidence Interval	Range likely containing true effect with $1-\alpha$ confidence.	If CI excludes 0 $\leftrightarrow$ significant. Wider CI = less power.
MDE (Minimum Detectable Effect)	Smallest effect size the test is powered to detect.	Trade-off: smaller MDE $\rightarrow$ larger N required.
Sample Size Formula	$n = 2\sigma^2(z_{\alpha/2}+z_\beta)^2 / \delta^2$	n per arm. σ^2=variance, δ=MDE. Run power analysis first.
Sequential Testing / SPRT	Continuously monitored tests with valid error bounds.	Fixes peeking problem. Use when can't wait for fixed N.
Novelty / Primacy Effects	Short-term behavioral changes due to newness of treatment.	Run experiment ≥2 weeks; separate new vs returning users.
CUPED (Controlled Exp.)	Variance reduction: $Y' = Y - \theta\cdot(X-E[X])$	Uses pre-experiment covariate to reduce variance by 20–50%.
Network Effects / SUTVA	Spillover: control users affected by treatment.	Use cluster/geo-based randomization; check SUTVA violations.

4.5 Guardrail Metrics (Always Monitor)

- **Latency (P50, P95, P99):** P99 latency matters most for user experience. Never ship if P99 > SLA threshold.
- **Impression / Click Diversity:** Gini coefficient of items shown. Avoid filter bubbles.
- **Fairness Metrics:** Demographic parity, equalized odds, counterfactual fairness.
- **Coverage:** Fraction of catalog receiving impressions. Prevents monopolization by head items.
- **CTR / CR on Control:** Monitor that control arm is healthy throughout experiment.

SECTION 5 — System Tradeoffs: Latency, Cost, Scalability, Fairness

5.1 Latency vs Accuracy Tradeoff

Factor	Tension	Mitigation
Model Complexity	Deep NN > GBT > LR. Each layer adds latency.	Distillation: train large teacher, deploy small student. Knowledge distillation reduces params by 10–100x.
Feature Freshness	Real-time features (last 5 min behavior) are most predictive but add I/O latency.	Hybrid: pre-compute 95% of features in batch; only 5% real-time. Use feature store (Feast, SageMaker FS).
Retrieval Depth	Re-ranking 1000 candidates > 100 candidates but more accurate.	Cascade architecture: fast ANN retrieval (1000) → light scorer (100) → heavy ranker (10).
Quantization	INT8 inference is 2–4x faster with <1% accuracy loss.	Use TensorRT, ONNX, or torch.quantize_dynamic for production.
Caching	Cache embeddings, pre-computed scores. Stale but fast.	LRU cache for top-1% popular items. TTL based on item volatility.

5.2 Cost vs Performance Tradeoff

Cost Driver	Magnitude	Mitigation
GPU Inference	Full precision LLM = $$$. Each API call: $0.01–0.06 / 1K tokens.	Use quantized models (GGUF, AWQ). Deploy open-source alternatives. Batch inference where latency allows.
Training Compute	Full fine-tuning LLAMA-70B ≈ $10K+. Frequent retraining expensive.	LoRA / QLoRA: train adapters only. Reuse frozen base model. Schedule retraining on drift detection, not calendar.
Data Storage	Storing all user events = petabytes at Amazon scale.	Stratified sampling. Keep last 90 days hot, archive older. Compress with Parquet + Snappy.
A/B Test Opportunity Cost	Running failed experiments costs revenue in treatment arm.	Run power analysis. Set conservative MDE. Kill losing arms fast with multi-armed bandit.

5.3 Scalability Patterns

Pattern	Description
Two-Stage Architecture	Stage 1: fast ANN retrieval (FAISS/Pinecone) to get top-K candidates. Stage 2: expensive ranker on K candidates. Decouples scale from quality.

Asynchronous Feature Pre-computation	Pre-compute user/item embeddings offline. Store in low-latency KV store (DynamoDB, Redis). Fetch at serve time in <5ms.
Approximate Nearest Neighbor (ANN)	HNSW, IVF, ScaNN for billion-scale vector search. Trade 0.1% recall for 100x speedup. Use M=32, ef_construction=200 for HNSW.
Model Versioning & Shadow Mode	Deploy new model in shadow (receives traffic, no effect). Compare offline vs online metrics before full rollout.
Horizontal Scaling	Stateless model servers behind load balancer. Autoscale on QPS. Cache + shard feature stores by user_id hash.

5.4 Fairness & Bias Tradeoffs

Bias Type	Mitigation
Accuracy vs Fairness	Higher overall AUC may mask poor performance on minority groups. Require AUC-by-slice reporting for protected attributes.
Exploration vs Exploitation	Greedy exploitation maximizes short-term revenue but starves tail items. Thompson Sampling / UCB balances both.
Popularity Bias	Collaborative filtering amplifies already-popular items. Correct with IPS (Inverse Propensity Scoring) weighting in training.
Feedback Loops	Model drives traffic → traffic generates labels → model reinforces bias. Break with random exploration (epsilon-greedy or forced impressions).

SECTION 6 — Common Pitfalls: What Costs You the Offer

Pitfall	Root Cause & Fix
■ **Data Leakage**	Including features that encode the label. Example: 'refund_amount' in fraud model. Fix: feature audit, time-aware train/test split.
■ **Offline-Online Gap**	Great AUC offline but no lift online. Root causes: position bias in training data, distribution shift, feedback loops not modeled.
■ **Ignoring Class Imbalance**	Training on raw 0.1% fraud data → model predicts all negatives. Fix: use AUC-PR, class weights, SMOTE, threshold tuning.
■ **Wrong Metric for Business Goal**	Optimizing CTR when business cares about revenue. High CTR + low conversion = wasted impressions. Align ML metric to OKR.
■ **No Baseline Comparison**	Proposing a Transformer without mentioning that LightGBM + good features likely achieves 90% of the lift at 1% of the cost.
■ **Peeking in A/B Tests**	Checking p-value daily and stopping when $p < 0.05$ → alpha inflation. Use sequential testing (SPRT) or pre-commit to sample size.
■ **Cold Start Ignored**	New users/items have no history. Model returns random scores. Fix: content-based fallback, popularity-based priors, onboarding questions.
■ **Single-Point Evaluation**	Reporting mean NDCG without variance, without stratification by query type, without time-based degradation curves.
■ **Ignoring Serving Constraints**	Designing a 175B-parameter model for <100ms P99 latency. Always scope model size to latency SLA from the start.
■ **Not Addressing Feedback Loops**	Model trained on biased historical impressions perpetuates those biases. Need randomization, IPS corrections, counterfactual logging.

CASE STUDY 1

Recommendation System — Amazon Product Recs

Problem Statement

■ Real Interview Question (Amazon AS L6, reported 2023)

'You are tasked with improving Amazon's "Customers who bought this also bought" widget on the product detail page. It currently uses item-item collaborative filtering on co-purchase counts. Propose an ML redesign: define the objective, choose a model, explain evaluation, and design the A/B test.'

Variant also asked: 'Design a personalized homepage feed for Amazon. How do you handle new users with no history?' — Amazon Bar Raiser round, 2024.

Step-by-Step Structured Answer

Step 1 — Problem Framing

- Business metric: Revenue per visit, units ordered, long-term customer LTV
- ML task: Top-K item ranking for each user. Unit of prediction: (user_id, candidate_item_id) pair
- Constraints: P99 < 100ms, cold-start for new users/items, catalog = 500M+ items

Step 2 — Data & Features

- **User features:** purchase/view history (last 90 days), demographic segment, session context, preferred categories, price sensitivity
- **Item features:** category, price, brand embedding, avg rating, review count, days since launch, stock status
- **Interaction features:** user-item co-views, brand affinity score, cross-category lift
- **Labels:** Implicit (purchase = 1, viewed-not-bought = 0.5, not shown = 0). Use BPR-style pairwise labels for ranking.
- **Data challenges:** 0.01% positive rate, position bias in historical clicks, new item cold start

Step 3 — Architecture (Two-Stage)

- **Stage 1 — Retrieval (candidate generation):** Two-Tower model. User encoder + Item encoder. Train with in-batch negatives + hard negatives. Approximate output: top 500 candidates. Deploy item embeddings in FAISS (IVF + HNSW). Latency: ~20ms.
- **Stage 2 — Ranking:** DCN-v2 (Deep & Cross Network). Input: user features + item features + cross features. Output: p(purchase | user, item). Top 10 from 500 candidates. Latency: ~30ms.
- **Cold-start fallback:** Content-based filtering using item metadata for new items. Popularity-based ranking for new users.
- **Diversity re-ranking:** Post-ranker applies MMR (Maximal Marginal Relevance) to diversify categories.

Step 4 — Training

- Loss function: BPR (Bayesian Personalized Ranking) for retrieval; Binary Cross-Entropy for ranker
- Correct for position bias using IPS (Inverse Propensity Scores) from randomized exploration
- Retrain retrieval weekly; ranker daily (purchase signal is abundant)
- Feature store: batch features pre-computed in Spark, served from DynamoDB; real-time features from Kinesis

Metrics

Offline	NDCG@10, Recall@50 (retrieval stage), AUC-PR (ranker)

Online (A/B)	Revenue per visit (+primary), CTR, Add-to-cart rate
Guardrails	Diversity (Gini@10 ≥ 0.6), Coverage, P99 latency < 100ms, new item impression rate
Long-term	30-day retention, Customer LTV, return visit rate

Tradeoffs

Relevance vs Diversity	Greedy ranking maximizes relevance but user sees 10 similar items. MMR re-ranking reduces NDCG@10 by ~1% but improves session depth.
Personalization vs Privacy	More behavioral signals = better recs. Balance with differential privacy, federated learning for sensitive signals.
Two-tower vs Direct Ranker	Two-tower scales to 500M items; direct ranker can't. But two-tower misses query-item interactions during encoding.
Freshness vs Stability	Daily retrain captures trends but introduces instability. Use exponential decay on old interactions.

■ WHAT A STRONG CANDIDATE SAYS

- I'd first establish whether we're optimizing for immediate purchase vs long-term LTV — these require different labels and loss functions.
- I'd use a two-stage architecture: fast ANN retrieval to 500 candidates, then an expensive DCN-v2 ranker. This gives us the quality of a deep model at the latency of a shallow one.
- The hardest problem isn't the model — it's correcting for position bias in training data using IPS, and handling the cold-start problem for the 10,000 new items added to Amazon's catalog daily.
- I'd run the A/B test with CUPED variance reduction and require 95% power on a 0.5% revenue MDE before launching.

CASE STUDY 2

Search & Ranking System — Amazon Product Search

Problem Statement

■ Real Interview Question (Amazon AS L6, reported 2023–2024)

'Amazon search currently ranks products using a combination of BM25 and hand-tuned business rules. You are asked to replace the ranking function with an ML model. Walk through your design: What is the training data? What model family? How do you handle position bias in click data? How do you evaluate offline vs online?' — Amazon Search team loop, 2024.

Variant: 'How would you improve search relevance for tail queries (<10 searches/day) where you have almost no behavioral data?' — Amazon Alexa Shopping, 2023.

Step-by-Step Structured Answer

Step 1 — Problem Framing

- Task: Learning to Rank (LTR). Given (query, document) pairs, predict a relevance score.
- Labels: Graded relevance (0=irrelevant, 1=related, 2=good, 3=perfect). Derived from purchase > add-to-cart > click > impression.
- Challenges: Query ambiguity ('apple' = fruit or device?), vocabulary mismatch, position bias, 1-5% queries are tail (never seen before).

Step 2 — Query Understanding

- **Query expansion:** Synonym expansion (ANN on query embeddings). Spell correction (noisy channel model).
- **Query classification:** Intent classifier → navigational / informational / transactional. Changes ranking strategy.
- **Entity recognition:** Extract brand, category, color, size from query string.

Step 3 — Multi-Stage Ranking Architecture

- **L0 — Inverted Index Retrieval:** BM25 match on title, bullets, category. Returns 10K–100K candidates. Latency: 10ms.
- **L1 — Fast Scorer:** LightGBM on TF-IDF features, title match score, brand match, price range. Prunes to 1K candidates. Latency: 15ms.
- **L2 — Neural Ranker:** BERT bi-encoder + tabular features. Fine-tuned on (query, product) pairs. Listwise LTR (LambdaRank / LambdaMART). Returns top 50. Latency: 40ms.
- **L3 — Business Rules Re-ranker:** Apply sponsored boosts, freshness penalties, in-stock filter, Prime eligibility boost. Returns final top 10.

Step 4 — LTR Training

- **Pointwise:** Predict relevance score per doc. Simple but ignores document interdependence.
- **Pairwise (RankNet, LambdaRank):** Predict which doc is more relevant. Directly optimizes ordering. LambdaRank uses NDCG gradients.
- **Listwise (ListNet, LambdaMART):** Optimizes full list. Best NDCG. LambdaMART = GBT + LambdaRank gradients. Production choice.
- **Debiasing:** Use random position swapping in 1% of traffic to collect unbiased relevance labels. Apply position-bias correction via Unbiased LambdaMART.

Metrics

Primary Offline	NDCG@5 (position-1 and 2 matter most in search), MRR (first relevant result)
Secondary Offline	Precision@1, Precision@5, Mean AP (MAP)
Online A/B	Revenue per search session, Purchase conversion rate, Zero-result rate
Guardrails	CTR on top-3 (no unintended demotion), query abandonment rate, P95 < 150ms
Tail Query Quality	NDCG on head / torso / tail query strata separately

Tradeoffs

BM25 vs Neural	BM25 is exact-match, fast, interpretable. Neural captures semantics but 10x slower and can hallucinate relevance.
Listwise vs Pairwise LTR	Listwise (LambdaMART) best NDCG but needs full query-level batches. Pairwise is simpler to implement.
Query Expansion vs Precision	Aggressive expansion increases recall but hurts precision (irrelevant results). Control with expansion confidence threshold.
Sponsored vs Organic	Sponsored results boost revenue but degrade relevance. Maintain separate organic slot for UX trust.

■ WHAT A STRONG CANDIDATE SAYS

- I'd treat this as a multi-stage LTR problem: fast BM25 retrieval → LightGBM L1 scorer → LambdaMART L2 neural ranker → business rule re-ranker. Each stage trades accuracy for throughput.
- The critical issue is position bias: historical clicks are biased toward position 1. I'd run a small random exploration (1% of traffic) to collect unbiased signal and apply Unbiased LambdaMART.
- For evaluation, I'd stratify NDCG@5 by query frequency: head queries (>1000/day), torso, and tail. Tail queries need special treatment — I'd fall back to content-based features since there's no behavioral history.
- I'd monitor zero-result rate and query reformulation rate as leading indicators of search quality degradation.

CASE STUDY 3

Fraud Detection — Transaction Risk Scoring

Problem Statement

■ Real Interview Question (Amazon Pay / AWS Fraud, reported 2022–2024)

'Design a real-time fraud detection system for Amazon Pay. You need to return a risk score for every transaction in under 50ms. Fraud rate is 0.1%. Walk through features, model choice, handling class imbalance, and how you would evaluate and monitor the system in production.' — Amazon Pay AS loop, 2023.

Variant asked at AWS: 'How would you detect account takeover (ATO) attacks where a fraudster logs in with stolen credentials? What signals distinguish legitimate login from ATO?' — AWS Security, 2024.

Follow-up commonly asked: 'Your model has 95% AUC-ROC but the business is unhappy. Why might that be, and what would you do?' — Tests understanding of AUC-PR vs AUC-ROC on imbalanced data.

Step-by-Step Structured Answer

Step 1 — Problem Framing & Business Context

• Two types of errors with asymmetric costs: FN (missed fraud) costs 100% of transaction + chargeback fee. FP (false alarm) costs customer friction + potential abandonment.

• Decision: Score → threshold → action. Threshold tuning is a business decision, not ML decision.

• Constraint: 50ms P99. Must be real-time. Rules-based baseline first, then ML on top.

Step 2 — Feature Engineering for Fraud

• **Velocity features:** # transactions in last 1/5/15/60 minutes by user, card, device, IP, merchant

• **Deviation features:** Amount vs user's historical mean (z-score), time-of-day deviation, location distance from usual

• **Graph features:** Connected components: is this device/IP linked to known fraud accounts? Fraud ring detection via GNN.

• **Behavioral biometrics:** Keystroke dynamics, mouse movement patterns, session navigation speed

• **Device fingerprint:** Browser/OS/font hash, VPN/proxy flag, headless browser detection, device age

• **Account signals:** Account age, email domain risk, # password resets last 30 days, linked card count

Step 3 — Modeling Strategy

• **Baseline:** Rule-based system (velocity rules, blocklists). Essential fallback. ~30% fraud catch at 0.5% FPR.

• **Model 1 — LightGBM:** On tabular features. Fast inference (< 5ms). Handles missing features. AUC ~0.94.

• **Model 2 — GNN (Graph Neural Network):** Capture fraud rings via network topology. Node features: transaction entities. Edge features: shared attributes (same IP, card). GraphSAGE or GAT.

• **Ensemble:** Calibrated blend of LightGBM + GNN + rules. Isotonic regression calibration for trustworthy probabilities.

• **Unsupervised:** Autoencoder for reconstruction error on new fraud patterns not in training data. Detects zero-day fraud.

Step 4 — Label Engineering & Class Imbalance

• Positive labels: Confirmed fraud (chargeback confirmed + manual review). NOT all chargebacks (some are legitimate disputes).

• Class imbalance (0.1%): Use class_weight='balanced' in LightGBM, or undersampling majority + SMOTE minority. Use AUC-PR not AUC-ROC.

• Label delay: Chargebacks arrive 30–90 days after transaction. Train on confirmed labels but add preliminary signals (dispute opened within 7 days).

• Adversarial: Fraudsters adapt. Retrain weekly minimum. Monitor KL-divergence of feature distributions for concept drift.

Metrics

Primary Offline	AUC-PR (most important for imbalanced data), KS Statistic, Precision@5% FPR
Business Metrics	Fraud catch rate (recall at operating threshold), $ fraud prevented, Chargeback rate
Cost Metrics	Expected cost = FN_cost × FN + FP_cost × FP. Minimize expected cost, not misclassification rate.
Online A/B	Shadow mode first: compare scores vs manual review outcomes for 30 days before live deployment
Monitoring	PSI (Population Stability Index) on score distribution. KL-divergence on feature distributions. Rule-based anomaly on score drift.

Tradeoffs

Recall vs Precision	Higher recall = catch more fraud but block more legit transactions. Business decision: what's FN cost vs FP cost? Typical operating point: 90% recall, 5% FPR.
Speed vs Sophistication	GNN is most powerful but takes 40ms. May exceed budget. Use LightGBM as primary, GNN for high-value transactions only (>$500).
Interpretability	Regulators may require explanation for declined transactions. SHAP values on LightGBM. Hard to explain GNN decisions.
Real-time vs Batch	Real-time scoring within the transaction. Batch scoring for account-level risk. Different latency SLAs.

■ WHAT A STRONG CANDIDATE SAYS

- I'd frame this as an expected cost minimization problem, not accuracy maximization. FN costs 110% of transaction value; FP costs $2 in customer support. This asymmetry drives my threshold choice.
- I'd use AUC-PR as the primary metric since AUC-ROC is misleading at 0.1% positive rate — a model that's right 99.9% of the time by always predicting negative looks great on ROC.
- The hardest part is concept drift: fraudsters are adversarial and adapt within weeks. I'd implement online learning with a sliding window and alert on PSI > 0.2 for key features.
- For graph features, I'd use GraphSAGE to detect fraud rings by propagating fraud signals across shared device/IP edges. This catches ~20% additional fraud that tabular models miss.

CASE STUDY 4

A/B Testing & Experiment Design

Problem Statement

■ Real Interview Question (Amazon AS — multiple teams, reported 2022–2024)

'You trained a new recommendation model with +3% NDCG@10 offline. Walk me through the A/B test you would run: how do you set sample size, what is your randomization strategy, how do you handle network effects, and what would make you decide to ship vs not ship?' — Amazon Personalization team, Bar Raiser round 2023.

Variant (Amazon Advertising, 2024): 'Your A/B test shows p=0.03 and a +0.8% CTR lift after 5 days. Your manager wants to ship immediately. What do you tell them?'

Variant (Amazon Fresh, 2023): 'How would you measure the long-term impact of a recommendation change, given that short-term A/B tests may not capture substitution effects or cannibalization from other categories?'

Step-by-Step Structured Answer

Step 1 — Hypothesis & Metrics

- **H■:** New model has no effect on revenue per visit.
- **H■:** New model increases revenue per visit by ≥ 0.5% (our MDE).
- **Primary metric:** Revenue per visit (directly tied to business goal).
- **Secondary metrics:** CTR, Add-to-cart rate, Conversion rate, Session depth.
- **Guardrail metrics:** P99 latency, Error rate, CTR on control (stability check).

Step 2 — Sample Size Calculation

- Parameters: $\alpha = 0.05$, Power = 0.80, two-tailed test.
- Baseline revenue per visit: \$2.50. σ = \$8.00 (high variance due to large orders).
- MDE: δ = \$0.0125 (0.5% of \$2.50).

$$n = 2 \times \sigma^2 \times (z_{\alpha/2} + z_{\beta})^2 / \delta^2 = 2 \times 64 \times (1.96 + 0.84)^2 / (0.0125)^2 \approx 8.1M \text{ users per arm}$$

- With 50M DAU, splitting 10% of traffic → ~5M users per arm → 16 days to reach significance. Use CUPED to reduce variance by ~30%, cutting required time to ~11 days.

Step 3 — Randomization Strategy

- **Unit of randomization:** user_id (not session/cookie) to avoid within-user contamination.
- **Stratified randomization:** Ensure equal distribution of user tenure, Prime membership, device type across arms.
- **Network effects / SUTVA:** In recommendations, user A seeing rec doesn't affect user B's experience. SUTVA holds. If we were testing social features, we'd need cluster randomization.
- **Holdout:** 45% control / 45% treatment / 10% holdout (for long-term effect estimation).

Step 4 — Running the Experiment

- **Ramp-up:** 0.1% → 1% → 10% → 50% over 3 days. Monitor guardrail metrics at each step.
- **Novelty effect check:** Segment by user tenure. If new users drive all lift, it may be novelty. Monitor week 1 vs week 2 effect separately.
- **Interaction effects:** Check if treatment × device, treatment × Prime, treatment × query type show heterogeneous effects.
- **No peeking:** Do NOT check p-value until pre-specified sample size reached. If must monitor, use sequential testing (SPRT) with valid alpha spending.

Step 5 — Analysis & Decision

- Compute t-test or bootstrap confidence interval on revenue per visit delta.
- Apply CUPED: $Y' = Y - \theta(X - E[X])$ where X = pre-experiment revenue. Reduces variance by using pre-period as covariate.
- Check for Simpson's Paradox: aggregate positive but negative within segments?
- Decision criteria: $p < 0.05$ AND effect size $\geq$ MDE AND no guardrail regressions $\rightarrow$ SHIP.

Metrics

Statistical	p-value, 95% CI on ATE (Average Treatment Effect), Cohen's d effect size
Business	Revenue per visit Δ, Total revenue impact ($\Delta \times$ DAU $\times$ LTV horizon)
Practical Significance	Is the effect size worth the engineering + maintenance cost? \$0.01/visit × 50M DAU = \$500K/day.
Long-term	14-day holdout: does effect persist? 30-day: does it improve or decay?

Tradeoffs

Speed vs Confidence	Smaller MDE requires longer experiment. Prioritize speed with CUPED variance reduction; sacrifice with α=0.10 for exploratory features.
User experience during test	50% of users get potentially worse experience for 2 weeks. Ramp carefully; kill fast if guardrails fail.
Multiple Testing	Testing 10 metrics simultaneously: familywise error rate inflates. Apply Benjamini-Hochberg correction, or pre-register primary metric.
Interference	Marketplace two-sided effects: treatment users buying depletes inventory, affecting control users' prices. Use synthetic control or difference-in-differences.

■ WHAT A STRONG CANDIDATE SAYS

- An offline NDCG lift of 3% is promising but means nothing without a live experiment. Offline-online correlation is often weak in recommendation systems due to position bias and feedback loops.
- I'd use CUPED variance reduction to cut required experiment duration by ~30%. Pre-experiment revenue is a strong covariate that explains much of the variance.
- The peeking problem is the most common A/B testing mistake I see. I'd pre-register the sample size and primary metric before launching, and use sequential testing if we need early stopping.
- I'd segment the treatment effect by user cohorts to detect heterogeneous effects. A +3% average might be +10% for Prime users and -2% for casual browsers — very different shipping decisions.

CASE STUDY 5

LLM-Based System — Product Q&A; RAG Chatbot

Problem Statement

■ Real Interview Question (Amazon Alexa / AWS Bedrock, reported 2023–2024)

'Design a system where customers can ask natural language questions about a product on its Amazon listing page — e.g., "Does this laptop support dual monitors?" — and receive accurate, grounded answers. Walk through retrieval, generation, evaluation, and guardrails against hallucination.' — Amazon Alexa Shopping AS loop, 2024.

Variant (AWS Bedrock team, 2024): 'A customer wants to build an enterprise knowledge-base chatbot using AWS. Design the RAG architecture. How do you evaluate retrieval quality separately from generation quality? How do you prevent the LLM from making up information not in the documents?'

Common follow-up: 'Your faithfulness score is 94% (6% hallucination). How do you get it to 99%? What are the tradeoffs?' — Tests depth on guardrail strategies.

Step-by-Step Structured Answer

Step 1 — Problem Framing

• Task: Open-domain QA over a closed, structured knowledge base (product catalog). Not a generative chatbot — answers must be grounded.

• Success metric: Answer accuracy, hallucination rate, user satisfaction (thumbs up/down), purchase conversion after Q&A; interaction.

• Failure modes: Hallucinated specs (dangerous for customer trust), missing context answers, latency > 2s, PII leakage in retrieved context.

Step 2 — RAG Architecture

• **Ingestion pipeline:** Chunk product content — title + bullets as one chunk, each spec section as separate chunk, each review as separate chunk. Overlap: 10%. Max chunk: 512 tokens.

• **Embedding model:** Fine-tuned bi-encoder (BGE-large or E5-large) on product domain QA pairs. Better than off-the-shelf for technical vocabulary.

• **Vector store:** OpenSearch with k-NN plugin for hybrid BM25 + dense retrieval. Reciprocal Rank Fusion (RRF) to merge results.

• **Retrieval:** Encode question → ANN search → top 20 chunks → cross-encoder re-ranker → top 5 chunks → LLM prompt.

• **Generation:** Prompt: 'Answer the question using ONLY the provided context. If the answer is not in the context, say so.' Context = top 5 chunks. LLM: Claude 3 Sonnet or GPT-4o. Max output: 150 tokens.

• **Citation:** Return source chunk IDs so UI can highlight source text. Builds user trust.

Step 3 — Evaluation Framework

• **Retrieval quality:** Recall@K — does the correct source chunk appear in top-K? Hit Rate@5 target: > 85%.

• **Answer faithfulness:** NLI model (DeBERTa fine-tuned on NLI) checks if answer is entailed by retrieved context. Target: < 2% hallucination rate.

• **Answer relevance:** Does the answer address the question? Cosine similarity between question embedding and answer embedding. Or LLM-as-judge.

• **Context precision:** Of retrieved chunks, what fraction are actually relevant? High irrelevant context → LLM confused, longer prompt.

• **End-to-end:** Human evaluation sample: 200 questions, expert-labeled answers, compare system answer to gold. Accuracy = % correct answers.

• **Online metrics:** Thumbs up/down, follow-up question rate (indicates unanswered question), purchase conversion rate after interaction.

Step 4 — Guardrails & Safety

• **Hallucination prevention:** System prompt explicitly restricts to provided context. Temperature = 0 for factual queries. NLI faithfulness check post-generation.

• **PII filtering:** Scrub PII from retrieved reviews before injecting into prompt. Regex + NER-based filter.

• **Toxic content:** Input safety classifier (block prompt injection attempts). Output toxicity classifier.

• **Prompt injection defense:** Validate that retrieved content doesn't contain instructions. Sanitize context chunks.

• **Fallback:** If retrieval returns < 3 relevant chunks (below relevance threshold), return 'This information isn't available in the product details' rather than hallucinate.

Metrics

Retrieval	Hit Rate@5, MRR, Context Precision, Context Recall (RAGAS framework)
Generation Quality	Faithfulness (NLI entailment score), Answer Relevance, ROUGE-L vs gold answers
End-to-End	Accuracy on human-labeled eval set, Exact Match for factoid questions
Online	User satisfaction (thumbs up rate), Conversion lift vs no-Q&A; control, Escalation to human support rate
Safety	Hallucination rate (<2% target), PII leak rate, Prompt injection success rate (0% target)
Efficiency	P50/P95/P99 latency per component, Token cost per query, Cache hit rate

Tradeoffs

Retrieval depth K	Larger K = higher recall but larger prompt = higher cost, slower, more distraction for LLM. Optimal K = 5–10 after re-ranking.
Chunk size	Small chunks (128 tokens): precise retrieval but missing context. Large chunks (1024 tokens): richer context but noisy retrieval.
LLM size	GPT-4o: best quality, $0.015/1K output tokens. GPT-3.5-Turbo: 10x cheaper, 15% quality drop. Fine-tuned smaller model: lowest cost for narrow domain.
Faithfulness vs Fluency	Strict grounding (only context facts) may produce stilted answers. Slight relaxation improves fluency but increases hallucination risk.
Latency budget	Retrieval (~100ms) + Re-ranking (~200ms) + LLM generation (~800ms) = ~1.1s total. Optimize re-ranker with distilled cross-encoder.

■ WHAT A STRONG CANDIDATE SAYS

- The #1 risk in production RAG is hallucination. I'd implement a faithfulness check using an NLI model as a post-generation filter, and explicitly instruct the LLM to say 'I don't know' when context is insufficient — this is more trustworthy than a fluent wrong answer.
- Hybrid retrieval (BM25 + dense) consistently outperforms either alone, especially for technical product queries with exact model numbers. I'd use Reciprocal Rank Fusion to combine the two ranked lists.
- For evaluation, I'd use the RAGAS framework which provides automated metrics for retrieval and generation quality. But I'd also run weekly human eval on a 50-question golden set to catch systematic degradation.
- Chunk design is underrated — I'd treat each product section (specs, features, reviews) as separate chunks with metadata, allowing the retriever to preferentially surface specs chunks for specification questions.
- For cost management at scale, I'd cache embeddings of common queries (top 10K account for 40% of traffic), and use a smaller model (Haiku/3.5-Turbo) for simple factoid answers, escalating to full GPT-4o only for complex multi-hop reasoning.

REPORTED REAL AMAZON AS INTERVIEW QUESTIONS BANK

The following questions have been reported by candidates across Glassdoor, Blind, LeetCode Discuss, and direct community reports (2022–2024). Organized by topic.

Machine Learning Design (System Design Round)

Source / Team	Question
Amazon Search (2024)	Design an ML system to personalize search result ranking for each user. How do you balance relevance vs personalization? How do you prevent filter bubbles?
Amazon Recommendations (2023)	The 'Frequently Bought Together' widget uses co-occurrence counts. Replace it with an ML model. Define the training signal, model, and evaluation.
Amazon Fresh (2023)	Design a demand forecasting model for perishable grocery items. How do you handle promotions, seasonality, and new product cold start?
Amazon Advertising (2024)	Design a CTR prediction model for sponsored product ads. How do you handle advertiser cold start (new ads with no impressions)?
Alexa (2023)	Design an intent classification system for Alexa voice queries. Queries can be ambiguous ('play music' — which music?). How do you handle ambiguity?
AWS SageMaker (2024)	A customer has a churn prediction model with 0.91 AUC-ROC but the business says it's not useful. Debug the problem and propose fixes.
Amazon Logistics (2023)	Predict package delivery time for Amazon deliveries. Features, model, evaluation. How do you handle extreme weather events as outliers?
Amazon Prime Video (2024)	Design a content recommendation system. How is it different from product recommendations? How do you handle the explore-exploit tradeoff for new shows?

ML Fundamentals & Depth Questions

Source	Question
Amazon (multiple, 2023–24)	Explain the bias-variance tradeoff. How does it manifest in a GBT model? How do you tune it?
Amazon Search (2024)	What is the difference between AUC-ROC and AUC-PR? When would you use one over the other? Give a concrete example from your work.
Amazon Pay (2023)	Your fraud model shows 0.95 AUC-ROC. Your colleague says AUC-ROC is misleading here. Is she right? What metric would you use instead and why?
Amazon Ads (2024)	Explain how LambdaMART works. Why is it better than pointwise regression for ranking? What are its limitations?

Amazon (Bar Raiser, 2023)	You have a model in production. 3 months later, performance degrades. Walk me through your full debugging process. What do you check first?
AWS ML (2024)	Explain NDCG@K. Why divide by log(i+1)? What happens if all your test queries have only one relevant document?
Amazon (2023)	How does gradient boosting differ from random forests? When would you choose one over the other?
Amazon Alexa (2024)	Explain the attention mechanism in Transformers. How does self-attention scale with sequence length? What is the practical implication?
Amazon (2024)	What is label leakage? Give 3 examples of how it can occur in a real ML pipeline. How do you detect it?
Amazon (Bar Raiser, 2024)	You run an A/B test for 3 days and see p=0.02. Your PM wants to ship. What do you do? What additional information do you need?

LLM & GenAI Questions (Increasingly Common Post-2023)

Source	Question
AWS Bedrock (2024)	Compare RAG vs fine-tuning for a customer support chatbot that needs to answer questions about a company's internal policies. Which would you choose and why?
Amazon (2024)	How do you evaluate a RAG system? What metrics would you use for (a) retrieval quality and (b) generation quality separately?
Amazon Alexa (2024)	Your LLM-based assistant hallucinates product specifications 8% of the time. Walk me through how you would reduce this to under 1%.
AWS (2024)	Explain the difference between semantic search and keyword search. When does semantic search fail? Give an example where BM25 would outperform a dense retrieval model.
Amazon (2024)	A product manager asks you to fine-tune an LLM to summarize customer reviews. What data do you need? What evaluation metrics? What could go wrong?
AWS Bedrock (2024)	What is RLHF? Explain the three stages. What is DPO and how does it simplify the RLHF pipeline?

Experiment Design Questions

Source	Question
Amazon (2023)	How do you design an A/B test when your treatment affects both buyers and sellers on a two-sided marketplace? (Interference / SUTVA violation)
Amazon (2024)	What is the difference between a p-value and a confidence interval? Which is more useful for a business decision? Why?
Amazon Ads (2023)	You want to test 5 ad ranking algorithms simultaneously. How do you control the familywise error rate? What is Bonferroni correction and when is it too conservative?

Amazon (2023)	Explain CUPED. How does it reduce variance? What is the covariate you would use for an Amazon purchase rate experiment?
Amazon Prime (2024)	How would you detect if your A/B test has a novelty effect? How long should you run the experiment to be sure?
Amazon (Bar Raiser)	Your A/B test shows +2% revenue but -0.5% in customer satisfaction score. How do you make the ship decision?

QUICK-REFERENCE CHEAT SHEET

Model Selection Cheat Sheet

Problem Type	Model Choice
Tabular, binary classification	LightGBM / XGBoost → Logistic Regression baseline
Sparse user-item matrix	ALS / BPR / LightFM → Two-Tower for scale
Dense ranking with features	DCN-v2 / LambdaMART
NLP classification/NER	Fine-tuned BERT / RoBERTa
Text generation / Q&A;	LLM + RAG (GPT-4o, Claude, Llama) with grounding
Real-time anomaly detection	Isolation Forest + LightGBM ensemble
Time-series forecasting	LightGBM (lag features) → Temporal Fusion Transformer
Image classification	ResNet50 / EfficientNet-B4 → ViT for SOTA
Graph fraud/network	GraphSAGE / GAT on transaction/entity graph
Multi-armed bandit (explore)	Thompson Sampling (Bayesian) / LinUCB (contextual)

Metric Quick-Select

Use Case	Preferred Metric(s)
Imbalanced classification (<5% pos)	AUC-PR, F1, MCC (not Accuracy, not AUC-ROC)
Balanced classification	AUC-ROC, F1, Accuracy
Ranking / search	NDCG@K, MAP, MRR, Recall@K
Recommendation (implicit feedback)	NDCG@10, Hit Rate@10, Coverage, Novelty
Regression / forecasting	RMSE (if outliers matter), MAE (robust), MAPE (scale-free)
Fraud / medical detection	Recall at fixed FPR, AUC-PR, Expected Cost
LLM generation quality	ROUGE-L (overlap), BERTScore (semantic), RAGAS (RAG-specific)
A/B test	ATE with 95% CI, p-value (pre-registered), Cohen's d effect size
Calibration	Expected Calibration Error (ECE), Brier Score

The Amazon Interview Mindset

■ **Senior Applied Scientist Signals**

Start with the business metric — always connect ML choices to revenue/retention/cost.

Propose baselines first — show you don't over-engineer. A logistic regression baseline is always required.

Quantify tradeoffs — don't say 'faster but less accurate'. Say '2x faster inference at 0.3% NDCG drop'.

Address failure modes proactively — cold start, concept drift, position bias, label delay.

Speak to scale — your solution must work at Amazon's scale: 500M products, 100M users, 50K QPS.

Close the loop — every design ends with a monitoring plan and experiment design for production validation.